

USER JOURNEY · V1.2

From Register to *Check-in*

The end-to-end attendee flow on the AI Salon Tel Aviv platform — from the moment a prospect lands on the public event page, through community engagement between members, to the moment door staff verify their entry code.

Engineering

Operations

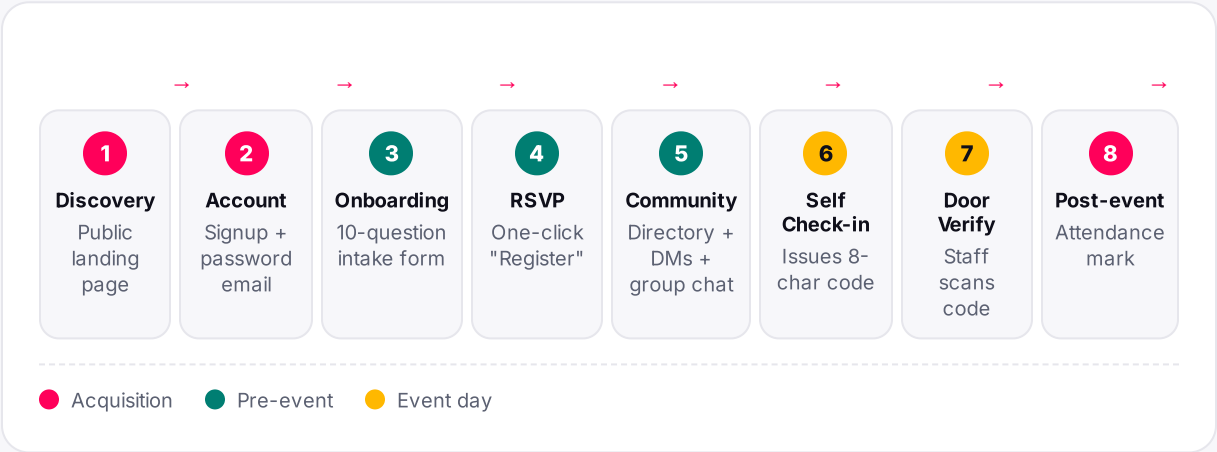
Mixed Audience

7-Stage Flow

The Journey at a Glance

Registration on this platform is a **one-click RSVP** for signed-in users — there is no long registration form. Account creation captures only a name and email; the rich profile is filled later via a forced onboarding intake. Once onboarded, members gain access to a **community directory** where they can browse other members and start 1-on-1 direct-message conversations, plus event-scoped group chat rooms. Check-in is split into two distinct phases: the attendee **self-checks-in** to generate a globally-unique 8-character code, then door staff **verify that code** at the venue in an explicit two-step lookup-and-confirm flow. The whole journey is automated by an email orchestrator that fires on RSVP, attendance, and door-checkin triggers.

Seven-Stage Flow Diagram



02 · ACQUISITION & ONBOARDING

Stages 1–3: Discovery, Account, Registration

STAGE 01 Discovery & Landing

/e/[slug]

A prospect arrives at the **public event page** at `/e/[slug]`. The page renders hero, speakers, agenda, and a sticky CTA card that adapts to four states: anonymous, signed-in-without-RSVP, registered-but-not-checked-in, and checked-in. If the visitor arrived via a member's share link, a `ais_utm_uid` cookie is set for later referral attribution.

CTA STATES 4 (anonymous → registered → checked-in → door-verified)

AUTH None required for viewing; required for RSVP

STAGE 02 Account Creation

POST /api/auth/signup

Clicking "Join AI Salon" routes to `/login?callbackUrl=/e/[slug]`. The signup form captures **only name and email** — no company, no phone, no dietary. The endpoint generates a random 8-character password (bcrypt-hashed at cost 10), creates a `User` row with `role=MEMBER`, generates a unique `utmUid` for referral tracking, writes a `ReferralAttribution` row if the UTM cookie is set, and emails the one-time password to the user. Receiving that email IS the verification — there is no separate confirmation step.

CAPTURED `email, name, passwordHash, role=MEMBER, utmUid`

EMAIL SENT "Your AI Salon Tel Aviv login" (plaintext password)

STAGE 03 Onboarding + RSVP

/onboarding → /api/events/[slug]/rsvp

On first login, because `onboardedAt IS NULL`, the user is bounced to a 10-question intake form (company, title, mobile, LinkedIn, interests, profile categories, apply-for, bio). Posting it sets `onboardedAt=now()` and redirects to `/events`. Back on the event page, the CTA now reads "Register to event". Clicking fires the RSVP endpoint, which is **idempotent** on `@@unique([eventId, email])` — duplicate clicks just upgrade status to GOING.

Only on the **first** registration does the endpoint send a confirmation email with an `.ics` calendar attachment and fire the `RSVP_GOING` trigger that enqueues any matching email-flow steps.

// RSVP UPSERT — THE HEART OF REGISTRATION

```
const rsvp = await db.eventRsvp.upsert({
  where: { eventId_email: { eventId: event.id, email: user!.email } },
  create: { eventId, userId, email, name,
    status: "GOING", source: "EVENT_PAGE", referredByUserId },
  update: { userId, status: "GOING" },
});
```

Stage 4: Community Directory & Messaging

STAGE 04

Community Engagement

/community · /events/[slug]/tabs/chat

Once onboarded, a member lands on `/community` — a paginated, searchable grid of every other onboarded member on the platform (card shows photo, name, title, company, profile categories, and a short bio excerpt). Each card has a **Contact** button that opens the messages dialog and either continues an existing 1-on-1 conversation or starts a new one. Concurrently, every event the member has RSVP'd to exposes a **group chat room** tab (Socket.io sidecar) where all registered attendees can collaborate in real time before, during, and after the event.

Direct messages live in the `DirectMessage` table (senderId, recipientId, body, readAt). Group chat lives in `ChatRoom` + `ChatMessage` tables, scoped per event. Both flows are pushed live over a WebSocket sidecar (Socket.io) so recipients see messages without polling — the previous 2-second poll storm was removed for sub-100ms perceived latency. Read receipts, unread counts, and "minimized vs. open" dialog state are all tracked client-side and reconciled with the server on dialog open.

MEMBER BROWSE	<code>/community</code> — search + filter by category, paginated grid
1-ON-1 DM	<code>/api/messages/[userId]</code> GET history / POST new — opens from Contact button
EVENT GROUP CHAT	<code>/api/chat/events/[eventId]/room</code> — auto-created per event, all RSVP'd members
REAL-TIME	Socket.io sidecar (mini-services/chat-service) — replaces poll storm

PATTERN · STALE-JWT SELF-HEAL (V1.2 RELIABILITY FIX)

The original JWT callback in `src/lib/auth.ts` set `token.id` once at login, with a fallback chain that could write the Google OAuth `sub` (NOT a Prisma UUID) into the cookie if the initial DB lookup failed transiently. Once that bad value was cached, every subsequent `db.user.findUnique({ where: { id: session.user.id } })` returned null and the Contact button surfaced **"User not found"** — even for known members the user had previously messaged. v1.2 introduces a 3-layer defense: (1) `src/lib/session-user.ts` exports `getMeId(session) / getMe(session, select)` that verify `session.user.id` against the DB on every API call and fall back to an email lookup if stale; (2) the JWT callback self-heals on subsequent requests via a new `token.idResolved` flag that re-resolves from DB by email; (3) the messages dialog now translates raw API errors into friendly toast messages ("Your session is stale — please refresh"). The same `getMeId` helper is now used by `/api/messages/[userId]`, `/api/messages/conversations`, and `/api/messages/unread-count`.

Stages 5–7: Codes, Door, Post-event

STAGE 05

Self Check-in (Code Issuance)

POST /api/events/[slug]/check-in

Starting **2 hours before** `startsAt` and up to 6 hours after `endsAt`, the CTA flips to "I'm here — Check in". Clicking validates the window and generates an **8-character Crockford base32 code** (alphabet `0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ` — no I/L/O/U to avoid 1/0/V confusion), formatted as `XXXX-XXXX`. The code is **globally unique** across all events (a single code identifies one RSVP platform-wide), idempotent on re-click, and stored on `EventRsvp.checkInCode` + `checkedInAt`.

Unregistered members see only a "Register to event" button — the check-in CTA is hidden until they RSVP. The server-side endpoint still uses `upsert` as a defensive fallback (creates an RSVP on the fly if a user hits the API without one), but the UI no longer exposes that path. Members must register first, then the check-in button appears in the registered state once the 2-hour window opens.

```
// TIME WINDOW: STARTSAT - 2H → ENDSAT + 6H
```

```
function isWithinCheckInWindow(startsAt, endsAt, now = new Date()) {  
  const open = startsAt.getTime() - 2 * 60 * 60 * 1000;  
  const close = endsAt.getTime() + 6 * 60 * 60 * 1000;  
  return now.getTime() ≥ open && now.getTime() ≤ close;  
}
```

STAGE
06

Door Check-in (Two-Step Confirm)

/admin/check-in · GET /lookup · POST /confirm

At the venue, door staff type or scan the code at `/admin/check-in`. The flow is now **two explicit steps** — no pre-approval gate. **ANY Super Admin, Admin, or Co-host of the event** can perform both steps at the door; the confirmation itself IS the approval.

Step 1 — Look up (GET `/api/admin/check-in/lookup?code=XXXX-XXXX`): normalizes input (`abcd1234` , lowercase, leading spaces all become `ABCD-1234`) and returns one of three outcomes: **MISS** (404, red panel), **PENDING_CONFIRM** (200, amber "Confirm member to check in" panel with member info + a red non-transferrable-code warning), or **ALREADY_USED** (200, amber "Code already used" panel with original check-in time + checker name). **No write happens here** — pure read.

Step 2 — Confirm (POST `/api/admin/check-in/confirm` with `{ code }`): the door staffer reviews the member info + the warning ("Personal, non-transferrable code. This code belongs to X only. Verifying it admits this person into the event — do not confirm if the attendee at the door is not the person shown above."), then presses "Confirm check-in". The endpoint atomically sets `doorCheckedAt` + `doorCheckedBy` using race-safe `updateMany({ where: { id, doorCheckedAt: null } })`. Returns **CONFIRMED** (200, green panel) on first check-in, or **ALREADY_USED** (200, amber panel) if another staffer beat us to it (`count === 0` → re-fetch + return original timestamp). After each cycle, the input clears and refocuses for the next attendee.

NO PRE-
APPROVAL

Confirmation at the door IS the approval — any Admin/CoHost can do it

TWO-STEP

Look up → review member + warning → confirm (race-safe atomic write)

SINGLE-USE

`doorCheckedAt` set once; subsequent scans return "already used"

STAGE
07

Post-event Attendance Mark

PATCH
`/api/admin/rsvps/[id]/attendance`

After the event, an admin marks each RSVP as **ATTENDED**, **NO_SHOW**, or **CLEAR**. This is independent of door check-in — someone can enter the door and leave early (still marked no-show for content), or attend without a door scan (admin override). The endpoint sets `attendedAt` / `noShow` / `attendedMarkedBy`, fires GA4 + Meta CAPI conversion events, and fires `triggerFlowsForRsvp({triggerKind: "MARKED_ATTENDED"})` — which typically enqueues the Recap email sent 24 hours after the event.

API Endpoints & Email Flow

Endpoints Touched in the Journey

M	PATH	AUTH	SETS ON DB
POST	/api/auth/signup	None	User row, passwordHash, utmUid, ReferralAttribution
POST	/api/user/onboarding	Member	User.onboardedAt, profile fields
POST	/api/events/[slug]/rsvp	Member	EventRsvp (status=GOING), fires RSVP_GOING trigger
DEL	/api/events/[slug]/rsvp	Member	Deletes RSVP row, preserves any checkInCode
POST	/api/events/[slug]/check-in	Member	EventRsvp.checkInCode, checkedInAt
POST	/api/admin/events/[id]/rsvps/[rsvpId]/approve	Admin/CoHost	approvedByCoHostId, approvedAt — optional vetting field, no longer gates door check-in
GET	/api/admin/check-in/lookup?code=...	Admin/CoHost	Read-only — returns PENDING_CONFIRM / ALREADY_USED / MISS (no write)
POST	/api/admin/check-in/confirm	Admin/CoHost	doorCheckedAt, doorCheckedBy (first confirm only, race-safe)
PATCH	/api/admin/rsvps/[id]/attendance	Admin	attendedAt/noShow/attendedMarkedBy, fires MARKED_ATTENDED
GET	/api/messages/conversations	Member	List of all DM conversations for current user (via getMeId)
GET	/api/messages/[userId]	Member	DM thread with target user (verifies both users exist)
POST	/api/messages/[userId]	Member	Send DM — creates row in DirectMessage, pushes via Socket.io
GET	/api/messages/unread-count	Member	Unread DM count for header badge (parallelized with conversations)
GET	/api/chat/events/[eventId]/room	Member	Resolve or auto-create the group chat room for an event
GET	/api/chat/rooms/[roomId]/messages	Member	Group chat history (paginated, before-cursor supported)
POST	/api/chat/rooms/[roomId]/messages	Member	Post to group chat — broadcast via Socket.io to room subscribers
POST	/api/chat/rooms/[roomId]/read	Member	Mark room as read up to a cursor — clears unread badge

Email Flow Integration — Two Parallel Systems

A dedicated deep-dive on the email system — including the 5-stage sequence, audit queries to verify a real send vs. demo data, env vars, and a 9-step verification checklist — is published separately at /email-system-architecture.pdf. The summary below is the cross-reference for this journey.

LEGACY

5-Stage Orchestrator

Time-relative to event.startsAt. Processed by a cron tick on /api/cron/email/send-scheduled.

- Awareness — 240h before
- Reminder — 48h before
- Final Prep — 4h before
- Day-Of — at start
- Recap — 24h after

FLOWS V2

Per-Step Triggers

Each EmailFlowStep has its own triggerKind + delay. Idempotent on (flowStepId, rsvpId).

- Each step independently triggered
- A/B subject variants supported
- Audience filter per step
- Writes to same EmailQueue table

Trigger Kinds & Where They Fire

RSVP_GOING

Fires from /api/events/[slug]/rsvp on first registration only.

DOOR_CHECKED_IN

Defined but never fired — see anomaly below.

MARKED_ATTENDED

Fires from attendance route on transition to ATTENDED.

MARKED_NO_SHOW

Fires from attendance route on transition to NO_SHOW.

MANUAL

Admin-triggered from the flow reports dashboard.

ANOMALY · MEDIUM SEVERITY

The DOOR_CHECKED_IN trigger is defined in flow-trigger.ts and branch rules reference it, but the door confirm endpoint (/api/admin/check-in/confirm/route.ts) never calls triggerFlowsForRsvp({triggerKind: "DOOR_CHECKED_IN"}) . Any flow step configured with this trigger will silently never fire. **Fix:** add the call after the updateMany succeeds (i.e. when updateResult.count === 1 and the response status is CONFIRMED), so re-scans of an already-used code don't re-fire the trigger.

06 · THE FOUR TIMESTAMPS

EventRsvp Field Reference

The EventRsvp table is the central record of this journey. Four timestamps together describe the complete lifecycle of an attendee's relationship to one event. Understanding who sets each one — and when — is essential for debugging door-checkin issues and reconciling post-event attendance reports.

FIELD	SET BY	WHEN
<code>checkedInAt</code>	The attendee	Clicks "I'm here" — issues the 8-char code (window: startsAt-2h → endsAt+6h)
<code>approvedAt</code>	Co-host (optional)	Optional vetting field on the admin registrants page — no longer gates door check-in. The confirmation at the door IS the approval.
<code>doorCheckedAt</code>	Door staff (Admin/CoHost)	Set by POST <code>/api/admin/check-in/confirm</code> after the staffer reviews the member info + non-transferrable-code warning. Single-use, race-safe via <code>updateMany</code> where <code>doorCheckedAt: null</code>
<code>attendedAt</code>	Admin	Post-event attendance mark — independent of door check-in

Edge Cases Worth Knowing

EDGE CASE · LATE WALK-IN (API-ONLY FALLBACK)

The server-side check-in endpoint (`/api/events/[slug]/check-in`) still uses `upsert` and will create an RSVP on the fly (status=GOING, source=EVENT_PAGE) AND issue the code in one operation. However, **the UI no longer exposes this path** — unregistered members see only a "Register to event" button until they RSVP. The `upsert` remains as a defensive fallback for direct API calls. Note that this shortcut does NOT fire `RSVP_GOING` triggers — those flow steps will never be enqueued for the walk-in. This is arguably correct (no point sending a 10-days-before email at event time), but worth understanding when reconciling email queue data.

PATTERN · RACE-SAFE DOOR WRITE

When two door staffers confirm the same code within milliseconds of each other (common at busy events), the confirm endpoint uses `updateMany({ where: { id, doorCheckedAt: null } })` instead of `update()` . The first staffer's call returns `count: 1` and is treated as the confirmed check-in (returns `status: "CONFIRMED"`). The second staffer's call returns `count: 0` (because `doorCheckedAt` is no longer null), triggering a re-fetch that returns `status: "ALREADY_USED"` with the original timestamp + checker name. This pattern prevents race conditions without transactions. Note that the lookup step (GET) is a pure read and never writes, so two simultaneous lookups of the same code are both safe and idempotent.

EDGE CASE · STALE JWT AND "USER NOT FOUND" ON CONTACT

Before v1.2, a transient DB failure during login could write the Google OAuth `sub` (a non-Prisma value) into the JWT cookie as `token.id`. The cookie is long-lived, so every subsequent API call that did `db.user.findUnique({ where: { id: session.user.id } })` returned null and the Contact button errored with **"User not found"** — even though the recipient clearly existed. The fix is layered: API routes now go through `getMeId(session)` in `src/lib/session-user.ts`, which verifies the JWT id against the DB and falls back to an email lookup if stale; the JWT callback self-heals on subsequent requests via a `token.idResolved` flag; and the messages dialog translates raw API errors into friendly toast messages. Members who experienced this bug will see it self-resolve on their next page load — no manual cookie clearing required.



Seven stages, four *timestamps*, one code.

The flow splits cleanly: the platform handles acquisition (stages 1–3), community engagement (stage 4) bridges members before the event, the attendee owns their check-in code (stages 5–6), and door staff verify with a single atomic write (stage 6). Post-event attendance is a separate concept from door check-in.

KEY TAKEAWAYS

- Registration is a one-click RSVP — no long form
- Unregistered members see only "Register" — no check-in shortcut
- Onboarded members get a community directory + 1-on-1 DMs + event group chat
- All messaging is real-time via Socket.io sidecar — no polling
- Stale JWT `token.id` self-heals via `getMeId()` + JWT callback re-resolution
- The 8-char code is **globally unique** across all events
- Check-in window opens **2 hours before** the event starts
- Door check-in is two-step: look up → review + warning → confirm
- Any Admin/CoHost can confirm at the door — no pre-approval gate
- `doorCheckedAt` is single-use and race-safe
- Post-event attendance is independent of door check-in
- `D00R_CHECKED_IN` trigger is defined but never fires — wire it up

